To get specific fields of instance or relation, where findAll is model method:

```
  async findAllPosts() {
    return await this.findAll({
      attributes: ['id', 'title', 'isPublished', 'createdAt', 'userId'],
      include: [
        {
          model: require('../models').User,
          as: 'author',
          attributes: ['id', 'username'] // Only specific columns from user
        },
        {
          model: require('../models').Tag,
          as: 'tags',
          attributes: ['id', 'name'], // Only specific columns from tags
          // avoid to include any column from pivot table
          through: { attributes: [] }
        }
      ],
      order: [['createdAt', 'DESC']]
    });
  }
```

****

To get All fields of model and relation, where find by id is model method:

```
async findPostById(id) {
  return await this.findById(id, {
    include: [
      {
        model: require('../models').User,
        as: 'author',
        include: [
          {
            model: require('../models').Profile,
            as: 'profile'
          }
        ]
      },
      {
        model: require('../models').Tag,
        as: 'tags',
        through: { attributes: [] }
      }
    ]
  });
}
```

****

To get paginated result:

```
async findPublishedPosts(page = 1, limit = 10) {
  return await this.paginate(page, limit, {
    where: { isPublished: true },
    attributes: ['id', 'title', 'createdAt'],
    include: [
      {
        model: require('../models').User,
        as: 'author',
        attributes: ['id', 'username']
      }
    ],
    order: [['createdAt', 'DESC']]
  });
}
```

Where the paginate method is:

```
  async paginate(page = 1, limit = 10, options = {}) {
    const offset = (page - 1) * limit;
    const result = await this.model.findAndCountAll({
      ...options,
      limit,
      offset,
      distinct: true
    });

    return {
      data: result.rows,
      pagination: {
        total: result.count,
        page,
        limit,
        totalPages: Math.ceil(result.count / limit)
      }
    };
  }
```

To use or-operation, where Op.or from sequelize:

```
const { Op } = require('sequelize')
  // Create new user with profile
  async createUser(userData, profileData) {
    // Check if user already exists
    const existingUser = await User.findOne({
      where: {
        [Op.or]: [{username: userData.username}, {email: userData.email}]
      }
    });

    if (existingUser) {
      throw new Error('Username or email already exists');
    }

    console.log("User not found")

    // Create user with profile using transaction
    const transaction = await require('../models').sequelize.transaction();

    try {
```

```
      const user = await userRepository.create(userData, { transaction });

      if (profileData) {
        await require('../models').Profile.create(
          { ...profileData, userId: user.id },
          { transaction }
        );
      }

      await transaction.commit();
      return await userRepository.findUserById(user.id);
    } catch (error) {
      await transaction.rollback();
      throw error;
    }
  }
```

To delete the data from db:

```
  async delete(id, options = {}) {
    const instance = await this.findById(id);
    if (!instance) return false;

    await instance.destroy(options);
    return true;
  }
```

For bulk insert data:

```
  async up (queryInterface, Sequelize) {
    // Insert users
    const users = await queryInterface.bulkInsert('Users', [
      {
        username: 'john_doe',
        email: 'john@example.com',
        createdAt: new Date(),
        updatedAt: new Date()
      },
      {
        username: 'jane_smith',
        email: 'jane@example.com',
        createdAt: new Date(),
        updatedAt: new Date()
      }
```

```
    ], { returning: true });
}
```

****

To validate the request body, params, or query, we need these modules: npm i --save express express-validator

---

To create the validation for body:

```
const { body } = require('express-validator')

const userRegistrationValidation = [
  body('email')
    .isEmail()
    .normalizeEmail()
    .withMessage('Must be a valid email address'),

  body('password')
    .isLength({ min: 8 })
    .withMessage('Password must be at least 8 characters long')
    .matches(/\d/)
    .withMessage('Password must contain at least one number')
    .matches(/[A-Z]/)
    .withMessage('Password must contain at least one uppercase letter'),

  body('username')
    .trim()
    .isLength({ min: 3, max: 20 })
    .withMessage('Username must be between 3 and 20 characters')
    .matches(/^[a-zA-Z0-9_]+$/)
    .withMessage('Username can only contain letters, numbers, and underscores'),

  body('age')
    .optional()
    .isInt({ min: 13, max: 120 })
    .withMessage('Age must be between 13 and 120'),

  body('newsletter')
    .optional()
    .isBoolean()
    .withMessage('Newsletter must be a boolean value')
];
```

```
module.exports = { userRegistrationValidation }
```

To create sanitize request body with customSanitizer:

```
const { body } = require('express-validator')

const sanitizeExample = [
  body('username').trim().escape(), // Remove whitespace and escape HTML
  body('bio').trim().escape(),
  body('tags')
    .optional()
    .isArray()
    .withMessage("Tags must be array")
    .customSanitizer(value => {
      // Ensure tags is always an array
      return Array.isArray(value) ? value : [value];
    }),
    body('tags.*').isString().withMessage("Tag must be string"),
];

module.exports = { sanitizeExample }
```

To create async validation using custom:

```
const { body } = require('express-validator')
const asyncValidation = [
  body('email')
    .isEmail()
    .custom(async (value) => {
      // Simulate database check
      const userExists = await checkEmailExists(value);
      if (userExists) {
        throw new Error('Email already registered');
      }
      return true;
    }),

  body('referralCode')
    .optional()
    .custom(async (value) => {
      const isValid = await validateReferralCode(value);
      if (!isValid) {
        throw new Error('Invalid referral code');
      }
      return true;
    })
];
```

To create query, and param validation:

```
const orderUserValidation = [
    param('userId').isUUID().withMessage('Invalid user ID'),
    query('limit')
      .optional()
      .isInt({ min: 1, max: 100 })
      .withMessage('Limit must be between 1 and 100'),
    query('sort')
      .optional()
      .isIn(['asc', 'desc'])
      .withMessage('Sort must be asc or desc')
]

module.exports = { orderValidation, orderUserValidation }
```

To add validation with request value:

```
const { body } = require('express-validator')

const customValidation = [
  body('website')
    .optional()
    .custom((value) => {
      try {
        const url = new URL(value);
        return ['http:', 'https:'].includes(url.protocol);
      } catch {
        throw new Error('Must be a valid URL with http:// or https://');
      }
    }),

  body('birthdate')
    .optional()
    .isDate()
    .custom((value) => {
      const birthDate = new Date(value);
      const today = new Date();
      const age = today.getFullYear() - birthDate.getFullYear();
      if (age < 18) {
        throw new Error('Must be at least 18 years old');
      }
      return true;
    }),

  body('confirmPassword')
    .custom((value, { req }) => {
      if (value !== req.body.password) {
        throw new Error('Passwords do not match');
      }
      return true;
    })
];

module.exports = { customValidation }
```

To add conditional validation:

```
const { body } = require('express-validator')

const conditionalValidation = [
  body('accountType')
    .isIn(['personal', 'business'])
    .withMessage('Account type must be personal or business'),

  // Only require companyName if accountType is 'business'
  body('companyName')
    .if(body('accountType').equals('business'))
    .notEmpty()
    .withMessage('Company name is required for business accounts'),

  body('taxId')
    .if(body('accountType').equals('business'))
    .matches(/^\d{2}-\d{7}$/)
    .withMessage('Valid Tax ID required for business accounts')
];

module.exports = { conditionalValidation }
```